

**“AÑO DE LA ESPERANZA Y EL FORTALECIMIENTO DE LA
DEMOCRACIA”**

**SISTEMA DE GESTIÓN DE PROCESOS
DE WINDOWS IMPLEMENTADO
MEDIANTE ESTRUCTURAS
DINÁMICAS LINEALES EN DEV C++**

Docente: Rosario Delia Osorio Contreras

INTEGRANTES:

Flores Torres Elvis Augusto

Carrión Herquinio Kheynell Alessander

Laura Torres Diego Fernando



**Universidad
Continental**

HUANCAYO - JUNÍN - PERÚ

2026

1. Análisis del Problema

1.1. Descripción del Contexto

Los sistemas operativos modernos, como Windows 10/11, ejecutan decenas de procesos de forma simultánea. Por ejemplo, al abrir Google Chrome, Spotify, Discord y Visual Studio Code al mismo tiempo, el sistema debe administrar la memoria RAM, el tiempo de CPU y las prioridades de cada proceso. Sin una gestión correcta, el sistema puede volverse lento, bloquearse o perder rendimiento.

El presente proyecto tiene como finalidad simular, mediante estructuras de datos dinámicas lineales implementadas desde cero en C++ (DEV C++), el comportamiento básico de un administrador de tareas de Windows, abarcando el registro de procesos, la planificación del CPU y la gestión de memoria.

1.2. Identificación del Problema

Windows ejecuta múltiples aplicaciones simultáneamente y necesita:

- Mantener un registro actualizado de todos los procesos activos.
- Decidir qué proceso debe ejecutarse primero según su prioridad.
- Asignar y liberar memoria RAM de forma eficiente cuando los procesos inician o terminan.
- Persistir el estado del sistema para recuperarlo ante un reinicio o cierre inesperado.

1.3. Procesos Simulados

ID	Nombre del Proceso	Descripción
1001	Google Chrome	Navegador web principal
1002	Spotify	Reproducción de música en streaming
1003	Discord	Comunicación por voz y texto
1004	Visual Studio Code	Editor de código fuente
1005	Explorador de Archivos	Gestión del sistema de archivos
1006	Windows Defender	Antivirus y protección del sistema
1007	Actualización de Windows	Descarga e instalación de parches
1008	Descargas en segundo plano	Gestión de transferencias de archivos
1009	Zoom / Google Meet	Videollamadas y conferencias

2. Requerimientos del Sistema

2.1. Requerimientos Funcionales

N°	Función	Descripción
RF01	Registrar procesos	Ingresar nuevos procesos con ID, nombre, prioridad, estado y memoria utilizada.

RF02	Buscar procesos	Localizar un proceso por su ID o por su nombre.
RF03	Modificar prioridad	Actualizar el nivel de prioridad de un proceso activo.
RF04	Eliminar procesos	Retirar un proceso de la lista de activos cuando finaliza.
RF05	Organizar por prioridad	Encolar procesos según su nivel de prioridad para simular la planificación del CPU.
RF06	Administrar memoria	Asignar (push) y liberar (pop) bloques de memoria usando una pila dinámica.
RF07	Persistencia de datos	Guardar el estado del sistema en archivos TXT y cargarlo al iniciar.
RF08	Visualizar estado	Mostrar la lista de procesos, la cola del planificador y el estado de la memoria.

2.2. Requerimientos No Funcionales

N°	Categoría	Descripción
RNF01	Plataforma	Desarrollado en C++ con DEV C++; ejecutable en Windows 10 o superior.
RNF02	Hardware mínimo	Procesador Intel Core i3 o equivalente; 4 GB de RAM mínimo.
RNF03	Rendimiento	Las operaciones básicas (insertar, buscar, eliminar) deben completarse en menos de 2 segundos.
RNF04	Gestión de memoria	Uso eficiente de memoria dinámica; sin fugas de memoria (memory leaks) durante la ejecución.
RNF05	Mantenibilidad	Código fuente modular, comentado y con estructura clara que facilite futuras mejoras.
RNF06	Interfaz	Interfaz por consola (CLI), sin necesidad de entorno gráfico avanzado.
RNF07	Almacenamiento	Persistencia mediante archivos TXT de tamaño reducido, sin bases de datos externas.

3. Diseño de la Solución

Módulo	Estructura de datos	Función principal
Gestor de Procesos	Lista enlazada simple	Registro y consulta de todos los procesos activos
Planificador de CPU	Cola de prioridad dinámica	Ordenar y ejecutar procesos según su prioridad
Gestor de Memoria	Pila dinámica (LIFO)	Asignación y liberación de bloques de RAM

4. Estructura de Datos Implementadas y Justificación

4.1. Lista Enlazada

La lista enlazada simple fue elegida para el módulo de gestión de procesos porque permite almacenar un número variable de procesos sin necesidad de reservar memoria fija de antemano. Cada nodo se crea dinámicamente al registrar un proceso y se libera al eliminarlo, lo que evita el desperdicio de memoria.

Esta estructura es adecuada para el gestor de procesos porque:

- La inserción al final de la lista es sencilla y no requiere reorganizar toda la estructura.
- La búsqueda por ID o nombre recorre la lista nodo a nodo, lo que es aceptable para el tamaño esperado del sistema (menos de 50 procesos simultáneos).
- La eliminación de un nodo en cualquier posición se realiza redirigiendo el puntero del nodo anterior, sin mover datos.
- Refleja la forma en que Windows mantiene su tabla de procesos: una colección dinámica que crece y decrece a medida que los procesos se inician y terminan.

4.2. Cola de Prioridad Dinámica

La cola de prioridad dinámica fue elegida para el planificador porque permite modelar el comportamiento real de los planificadores de CPU de Windows, donde los procesos no se ejecutan en orden de llegada sino según su importancia o urgencia.

Por ejemplo, Windows Defender (alta prioridad) debe ejecutarse antes que Spotify (baja prioridad), aunque Spotify haya sido registrado primero. La cola de prioridad garantiza este comportamiento al insertar siempre en la posición correcta según el nivel de prioridad.

Esta estructura es adecuada porque:

- El encolamiento mantiene el orden de prioridad automáticamente en cada inserción.
- El desencolamiento siempre extrae el proceso de mayor prioridad (el del frente), simulando la ejecución del CPU.
- Es dinámica: no tiene límite fijo de capacidad, a diferencia de un arreglo estático.

4.3. Pila Dinámica

La pila dinámica fue elegida para el gestor de memoria porque sigue el principio LIFO (Last In, First Out), que refleja cómo los bloques de memoria del stack de un proceso se asignan y liberan: el último bloque asignado es siempre el primero en liberarse cuando el proceso termina.

Adicionalmente, la pila permite mantener un historial de procesos cerrados recientemente: si el usuario cierra primero Chrome, luego Discord y finalmente Spotify, al consultar el historial aparecerá primero Spotify (el último cerrado), luego Discord y finalmente Chrome.

Esta estructura es adecuada porque:

- Las operaciones push y pop son de tiempo constante $O(1)$, lo que garantiza una respuesta inmediata.

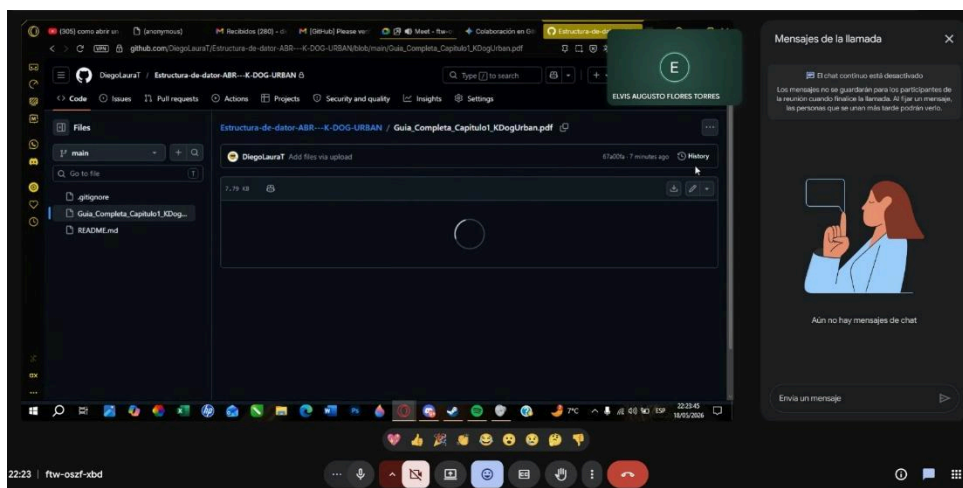
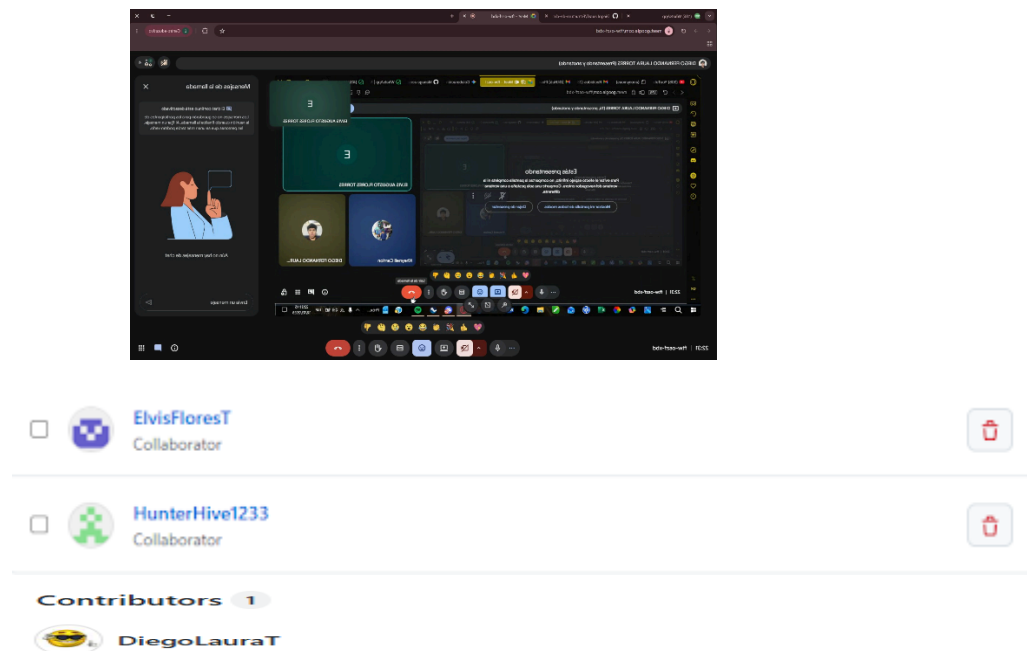
- Modela correctamente la liberación de memoria tipo LIFO que utiliza el stack de llamadas de cualquier proceso.
- Permite un historial de acciones que puede usarse para "deshacer" operaciones recientes.

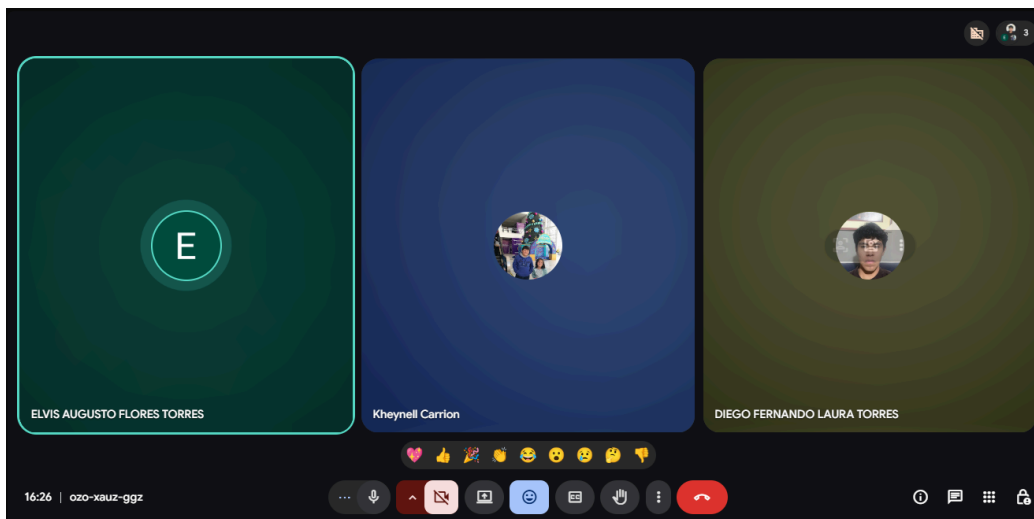
4.4. Justificación General de la Elección

La combinación de estas tres estructuras permite cubrir los tres aspectos fundamentales de la administración de procesos en un sistema operativo:

Necesidad del SO	Estructura elegida	Razón
Registro dinámico de procesos	Lista enlazada	Tamaño variable, inserción/eliminación eficiente en cualquier posición
Planificación por prioridad	Cola de prioridad	Garantiza que el proceso más urgente siempre se ejecute primero
Gestión de memoria RAM	Pila dinámica	LIFO refleja la asignación/liberación real del stack de procesos

5. Evidencias de trabajo Grupal:





Historial de versiones

Todas las versiones ▼

> 20 de mayo, 17:04



Versión actual

- DIEGO FERNANDO LAURA TORRES
- ELVIS AUGUSTO FLORES TORRES
- KHEYNELL ALESSANDER CARRIÓN
- HERQUINIO

2.3. Diapositiva – Canva:

<https://canva.link/dz7mkbqxj2k54gt>

2.3. Enlace GITHUB

<https://github.com/DiegoLauraT/ESTRUCTURA-DE-DATOS---EVALUACION---EQUIPO-5.git>